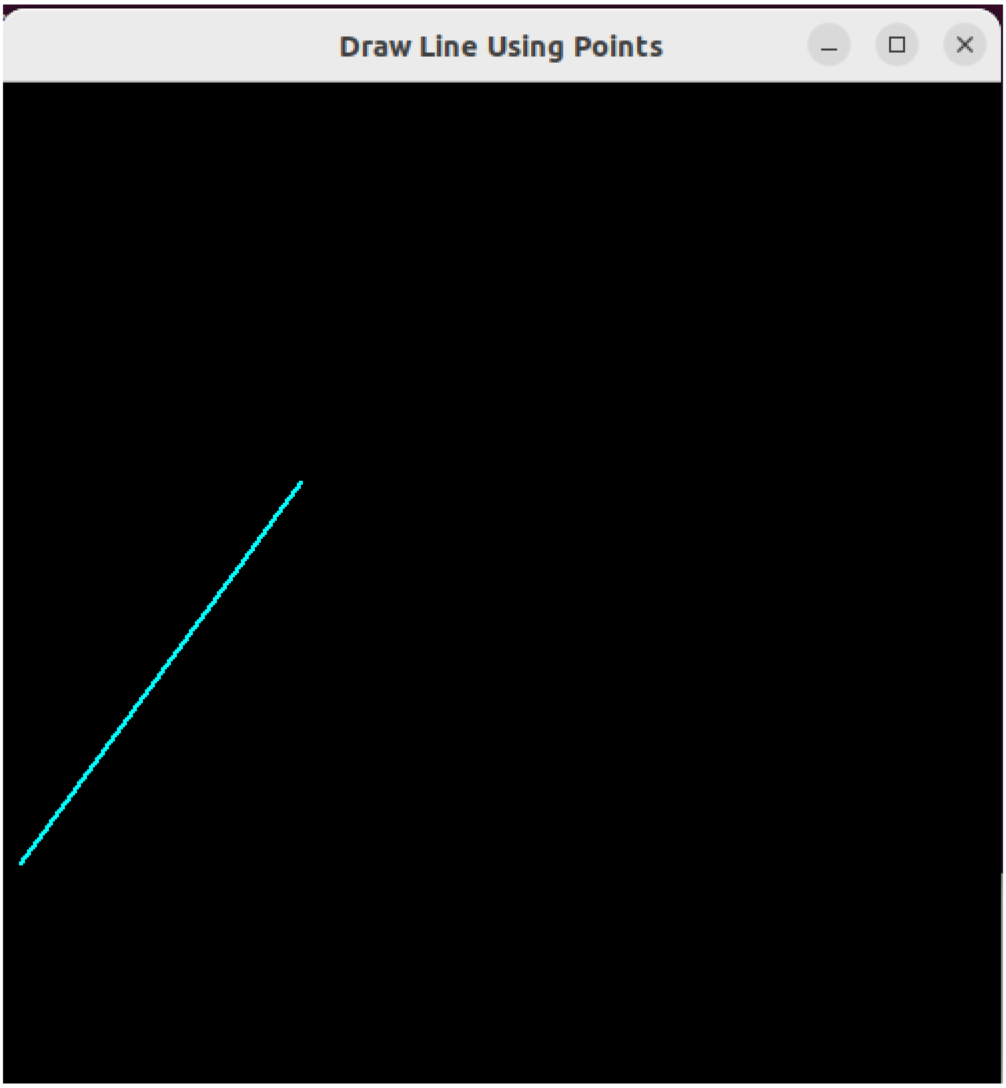
1. DDA Algorithm

```
#include <GL/glut.h>
#include <iostream>
#include <cmath>
using namespace std;

double x1 = 0.0, Y1 = 0.0;
double x2 = 0.0, Y2 = 0.0;

void init()
{
    glClearColor(0.0, 0.0, 0.0, 0.0);
    glMatrixMode(GL_PROJECTION);
    gluOrtho2D(0.0, 500.0, 0.0, 500.0);
}

void input()
{
    cout << "Enter the coordinates of the first point: ";
    cin >> x1 >> Y1;
    cout << "Enter the coordinates of the second point: ";
    cin >> x2 >> Y2;
}

void display()
{
    glClear(GL_COLOR_BUFFER_BIT);
    glColor3f(0.0, 1.0, 1.0);
    glPointSize(2.0);

    double x = x1;
    double y = Y1;

    double dx = x2 - x1;
    double dy = Y2 - Y1;

    double slope = fabs(dy / dx);

    int xStep = (dx >= 0) ? 1 : -1;
    int yStep = (dy >= 0) ? 1 : -1;

    glBegin(GL_POINTS);
```

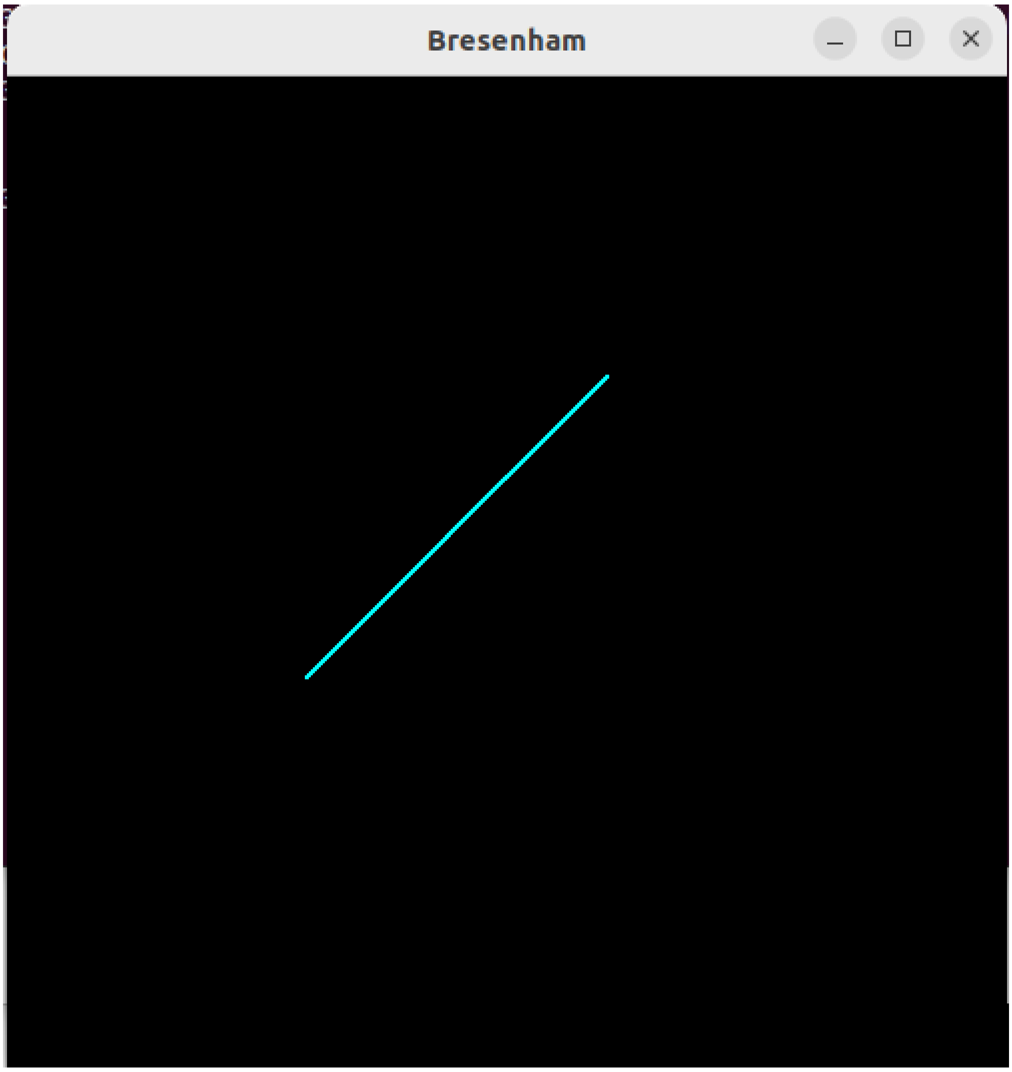
```
    if (slope <= 1)
    {
        while ((xStep > 0 && x <= x2) || (xStep < 0 && x >= x2))
        {
            glVertex2i((int)round(x), (int)round(y));
            x += xStep;
            y += slope * yStep;
        }
    }
    else
    {
        while ((yStep > 0 && y <= Y2) || (yStep < 0 && y >= Y2))
        {
            glVertex2i((int)round(x), (int)round(y));
            y += yStep;
            x += (1.0 / slope) * xStep;
        }
    }

    glEnd();
    glFlush();
}


int main(int argc, char **argv)
{
    input();
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
    glutInitWindowSize(500, 500);
    glutCreateWindow("Draw Line Using Points");
    init();
    glutDisplayFunc(display);
    glutMainLoop();
    return 0;
}
```


Draw Line Using Points

2. Bresenham's Line Drawing Algorithm

```
#include <GL/glut.h>
#include <iostream>
#include <cmath>
using namespace std;

double x1 = 0.0, Y1 = 0.0;
double x2 = 0.0, Y2 = 0.0;

void init()
{
    glClearColor(0.0, 0.0, 0.0, 0.0);
    glMatrixMode(GL_PROJECTION);
    gluOrtho2D(0.0, 500.0, 0.0, 500.0);
}

void input()
{
    cout << "Enter the coordinates of the first point: ";
    cin >> x1 >> Y1;
    cout << "Enter the coordinates of the second point: ";
    cin >> x2 >> Y2;
}

void display()
{
    glClear(GL_COLOR_BUFFER_BIT);
    glColor3f(0.0, 1.0, 1.0);
    glPointSize(2.0);

    double dy = Y2 - Y1;
    double dx = x2 - x1;
    double slope = (dy * 1.0 / dx);

    double p = (2 * dy) - dx;
    double x = x1;
    double y = Y1;
    glBegin(GL_POINTS);
```

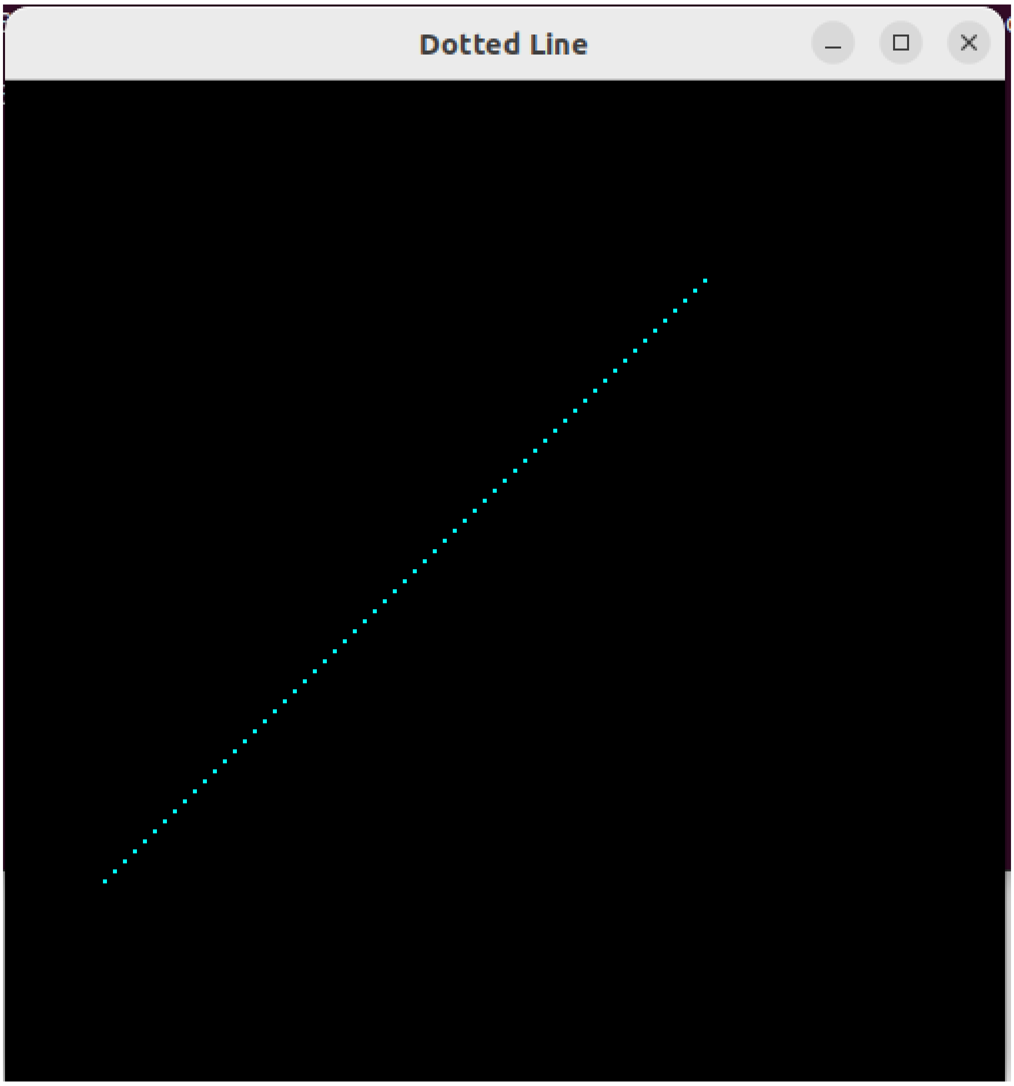
```
    while(x <= x2)
    {
        glVertex2i(x, y);
        if(p < 0)
        {
                p += 2 * dy;
        }
        else
        {
                y += 1;
                p += 2 * dy - 2 * dx;
        }
        x += 1;
    }

    glEnd();
    glFlush();
}


int main(int argc, char **argv)
{
    input();
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
    glutInitWindowSize(500, 500);
    glutCreateWindow("Draw Line Using Points");
    init();
    glutDisplayFunc(display);
    glutMainLoop();
    return 0;
}
```


Bresenham

3. Dotted Line

```
#include <GL/glut.h>
#include <iostream>
#include <cmath>
using namespace std;

double x1 = 0.0, Y1 = 0.0;
double x2 = 0.0, Y2 = 0.0;

void init()
{
    glClearColor(0.0, 0.0, 0.0, 0.0);
    glMatrixMode(GL_PROJECTION);
    gluOrtho2D(0.0, 500.0, 0.0, 500.0);
}

void input()
{
    cout << "Enter the coordinates of the first point: ";
    cin >> x1 >> Y1;
    cout << "Enter the coordinates of the second point: ";
    cin >> x2 >> Y2;
}

void display()
{
    glClear(GL_COLOR_BUFFER_BIT);
    glColor3f(0.0, 1.0, 1.0);
    glPointSize(2.0);

    double dy = Y2 - Y1;
    double dx = x2 - x1;
    double slope = (dy * 1.0 / dx);

    double p = (2 * dy) - dx;
    double x = x1;
    double y = Y1;
    int count = 0;
```

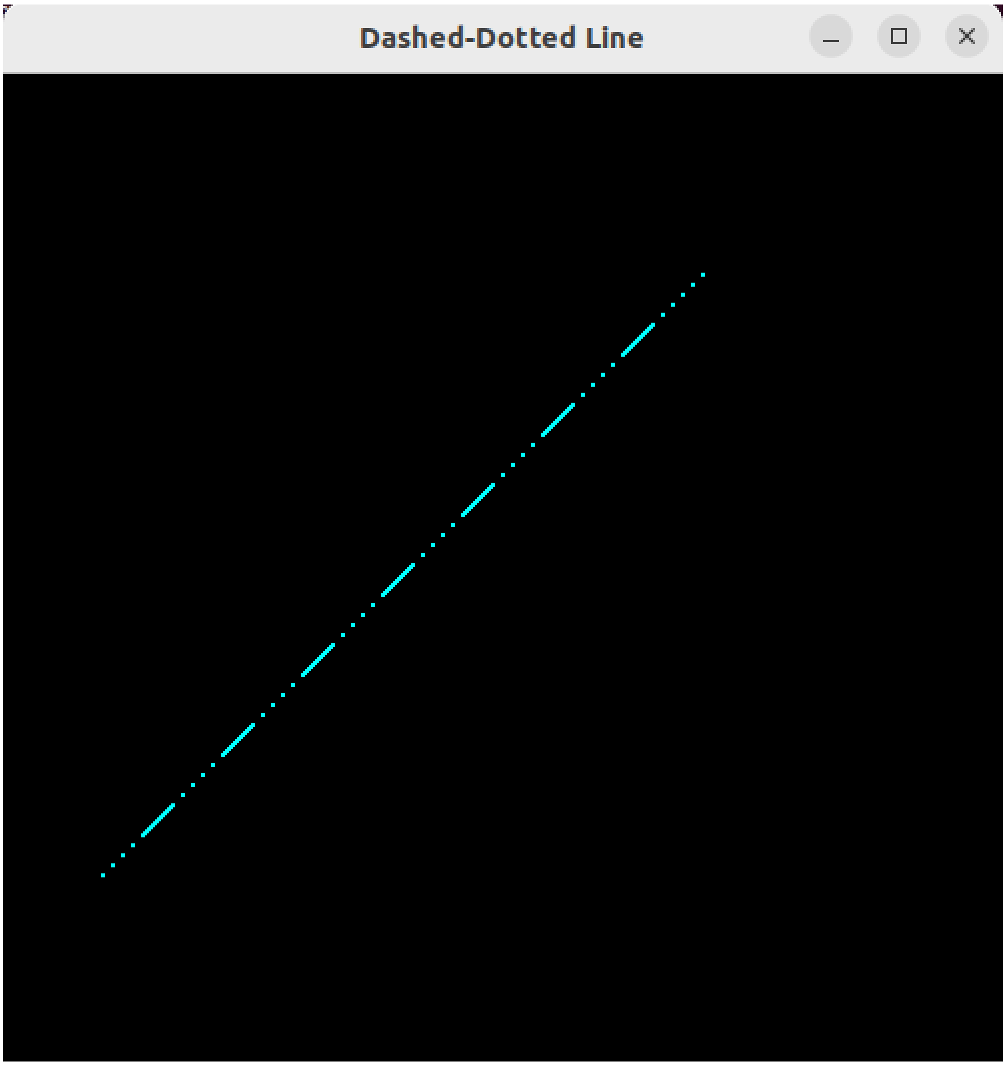
```
    glBegin(GL_POINTS);
    while(x <= x2)
    {
        if(count == 0)
        {
                glVertex2i(x, y);
                count = 5;
        }
        count--;
        if(p < 0)
        {
                p += 2 * dy;
        }
        else
        {
                y += 1;
                p += 2 * dy - 2 * dx;
        }
        x += 1;
    }

    glEnd();
    glFlush();
}


int main(int argc, char **argv)
{
    input();
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
    glutInitWindowSize(500, 500);
    glutCreateWindow("Draw Line Using Points");
    init();
    glutDisplayFunc(display);
    glutMainLoop();
    return 0;
}
```


Dotted Line

4. Dashed-Dotted Line

```
#include <GL/glut.h>
#include <iostream>
#include <cmath>
using namespace std;

double x1 = 0.0, Y1 = 0.0;
double x2 = 0.0, Y2 = 0.0;

void init()
{
    glClearColor(0.0, 0.0, 0.0, 0.0);
    glMatrixMode(GL_PROJECTION);
    gluOrtho2D(0.0, 500.0, 0.0, 500.0);
}

void input()
{
    cout << "Enter the coordinates of the first point: ";
    cin >> x1 >> Y1;
    cout << "Enter the coordinates of the second point: ";
    cin >> x2 >> Y2;
}

void display()
{
    glClear(GL_COLOR_BUFFER_BIT);
    glColor3f(0.0, 1.0, 1.0);
    glPointSize(2.0);

    double dy = Y2 - Y1;
    double dx = x2 - x1;
    double slope = (dy * 1.0 / dx);

    double p = (2 * dy) - dx;
    double x = x1;
    double y = Y1;
    int count = 0;
    bool flag = true; // true is for dotted, false is for dashed
    int dot_counter = 5;
    int dash_counter = 15;

    glBegin(GL_POINTS);
    while(x <= x2)
    {
        if(dot_counter > 0) {
                if(count == 0)
                {
                        glVertex2i(x, y);
                        count = 5;
                        dot_counter -= 1;
                        if(!dot_counter)
                                dash_counter = 15;
```

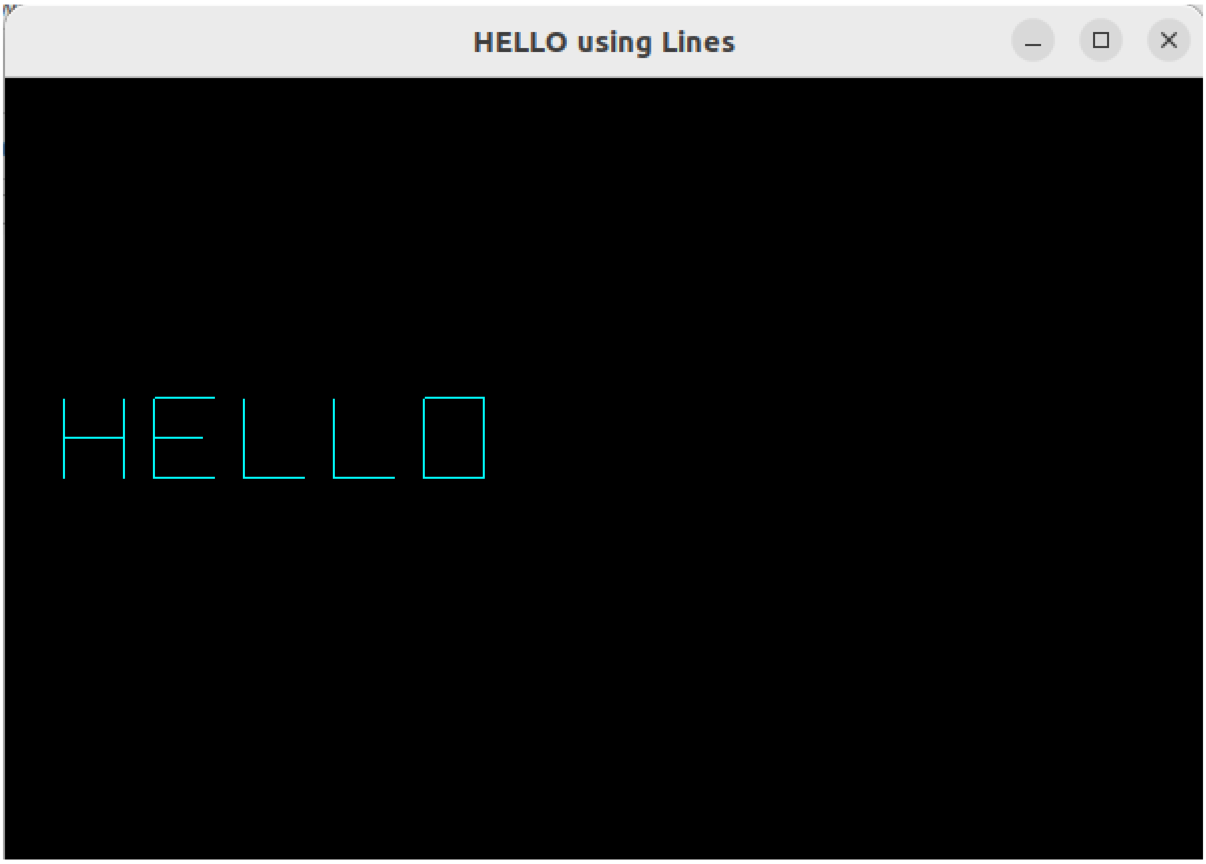
```
                count--;
                if(p < 0)
                {
                        p += 2 * dy;
                }
                else
                {
                        y += 1;
                        p += 2 * dy - 2 * dx;
                }
                x += 1;
        }
        else
        {
                glVertex2i(x, y);
                dash_counter -= 1;
                if(p < 0)
                {
                        p += 2 * dy;
                }
                else
                {
                        y += 1;
                        p += 2 * dy - 2 * dx;
                }
                x += 1;
                if(!dash_counter)
                        dot_counter = 5;

        }

    }

    glEnd();
    glFlush();
}


int main(int argc, char **argv)
{
    input();
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
    glutInitWindowSize(500, 500);
    glutCreateWindow("Dashed-Dotted Line");
    init();
    glutDisplayFunc(display);
    glutMainLoop();
    return 0;
}

```


Dashed-Dotted Line

5. Draw HELLO using lines.

```
#include <GL/glut.h>

void drawH(float x, float y)
{
    glBegin(GL_LINES);
        glVertex2f(x, y);      glVertex2f(x, y + 0.2);
        glVertex2f(x + 0.1, y); glVertex2f(x + 0.1, y + 0.2);
        glVertex2f(x, y + 0.1); glVertex2f(x + 0.1, y + 0.1);
    glEnd();
}

void drawE(float x, float y)
{
    glBegin(GL_LINES);
        glVertex2f(x, y);      glVertex2f(x, y + 0.2);
        glVertex2f(x, y + 0.2); glVertex2f(x + 0.1, y + 0.2);
        glVertex2f(x, y + 0.1); glVertex2f(x + 0.08, y + 0.1);
        glVertex2f(x, y);      glVertex2f(x + 0.1, y);
    glEnd();
}

void drawL(float x, float y)
{
    glBegin(GL_LINES);
        glVertex2f(x, y);      glVertex2f(x, y + 0.2);
        glVertex2f(x, y);      glVertex2f(x + 0.1, y);
    glEnd();
}

void drawO(float x, float y)
{
    glBegin(GL_LINES);
        glVertex2f(x, y);          glVertex2f(x, y + 0.2);
        glVertex2f(x, y + 0.2);    glVertex2f(x + 0.1, y + 0.2);
        glVertex2f(x + 0.1, y + 0.2); glVertex2f(x + 0.1, y);
        glVertex2f(x + 0.1, y);    glVertex2f(x, y);
    glEnd();
}
```

```
void display()
{
    glClear(GL_COLOR_BUFFER_BIT);
    glColor3f(0.0, 1.0, 1.0);

    float startX = -0.9f;
    float startY = 0.0f;
    float gap = 0.15f;

    drawH(startX, startY);
    drawE(startX + gap, startY);
    drawL(startX + 2 * gap, startY);
    drawL(startX + 3 * gap, startY);
    drawO(startX + 4 * gap, startY);

    glFlush();
}

void init()
{
    glClearColor(0, 0, 0, 1);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(-1, 1, -1, 1);
}

int main(int argc, char** argv)
{
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
    glutInitWindowSize(600, 400);
    glutCreateWindow("HELLO using Lines");
    init();
    glutDisplayFunc(display);
    glutMainLoop();
    return 0;
}

```


HELLO using Lines
HELLO